Threat Intelligence

7 min read

Snapshots & Slip-Ups in Nginx-UI and Snapd

Learn all about CVE-2026-27944 and CVE-2026-3888, two high-impact vulnerabilities affecting Nginx-UI and snapd, enabling unauthenticated data exposure and local privilege escalation.

pho3o Mar 23, 2026

CVE Explained

Some bugs are super obvious. Others just sit there, waiting to be noticed - usually by the wrong person.

In this case we've got a bit of both.

First, CVE-2026-27944 showcases a single publicly exposed backup endpoint in Nginx-UI that happily hands over everything you'd want - configs, credentials, even the keys to decrypt it all - no questions asked.

Then comes CVE-2026-3888 which is a bit more subtle. It doesn't give you access - it assumes you already have it. From there, it quietly turns a low-privileged foothold into full root escalation by abusing how snapd and system cleanup routines interact.

In this post, we're going to break down CVE-2026-27944 and CVE-2026-3888. We also introduce Snapped, a new HTB machine to offer a playground to explore how both these vulnerabilities can be chained and lead to full system compromise.

TL;DR

- CVE-2026-27944 (Nginx-UI) A publicly exposed /api/backup endpoint allows anyone to download full application backups along with the encryption key and IV needed to decrypt them. Result: instant access to sensitive configs, credentials, and TLS material.
- CVE-2026-3888 (snapd) A local privilege escalation vulnerability where a low-privileged user can abuse temporary directory cleanup and reuse to gain root-level execution on the host.

CVE-2026-27944 — Unauthenticated Backup Disclosure in Nginx-UI

Nginx-UI is an easy convenient solution to managing Nginx websites and configurations. Although it may be convenient to build and manage your website configurations, in versions before 2.3.3 it also offered a rather easy and convenient way for attackers to get complete access to those configurations.

CVE-2026-27944 is a critical vulnerability in Nginx-UI that allows unauthenticated attackers to retrieve and decrypt full application backups via the exposed /api/backup endpoint. It is classified as a low complexity unauthenticated vulnerability that allows Authentication Bypass and Sensitive Data Exposure.

How does it work?

1. Unauthenticated Access - CWE-306: The /api/backup endpoint is publicly accessible without an authentication check. Any user can send a simple HTTP GET request to the endpoint and trigger a full backup of both the Nginx and Nginx-UI configurations.

2. Missing Encryption of Sensitive Data - CWE-311: Nginx-UI backups are encrypted using AES-256 so you'd think that even if an attacker may get access to the backup archive they won't really be able to use it. However, the server also provides the decryption material in plaintext in the response header in the format X-Backup-Security: <Base64 Key>:<Base64 IV>. Thus, negating the purpose of encryption.

Affected Versions:

Nginx-UI: All versions prior to 2.3.3 are vulnerable.

What is Exposed?

The backup archives contain sensitive files such as:

- database.db: User credentials and session tokens for Nginx-UI
- app.ini: Nginx-UI configuration secrets
- nginx.conf: Nginx configuration secrets
- server.key/cert: SSL private keys

What is the Impact?

Successful exploitation grants full access to sensitive configuration data. The severity of this depends on the contents of the backup but can include:

- Credential & Session Exposure: Leading to administrative access of Nginx-UI and/or access to other accounts reusing credentials
- Configuration Disclosure: Revealing internal services and proxy rules
- Traffic Manipulation: Enabling malicious reconfiguration of routing behavior
- TLS Key Leakage: Allowing impersonation and interception of secure traffic

Mitigation

- Upgrade to Nginx-UI version 2.3.3 or later
- Restrict network exposure and access to Nginx-UI (restrict public exposure)
- Rotate all potentially exposed secrets, including credentials and TLS materials

CVE-2026-3888 — Local Privilege Escalation in Snap (snapd)

Following the Nginx-UI vulnerability which achieves full compromise of the management plane, CVE-2026-3888 can be chained to achieve full system compromise.

CVE-2026-3888 is a high-severity flaw affecting snapd. It allows an unprivileged local user to escalate privileges to root by abusing an interaction between two privileged standard system components: snap-confine (the component responsible for setting up snap sandboxes) and systemd-tmpfiles (which handles periodic cleanup of temporary directories).

How does it work?

At its core, the issue comes down to a timing and trust problem in how system directories are managed:

1. Trusted cleanup behavior is abused systemd-tmpfiles automatically removes

stale temporary directories after a certain period. One of these directories is used by snap-confine during sandbox initialization.

2. Race condition / timing window After the cleanup process deletes the directory, an attacker can recreate it and populate it with malicious files during the window before snap-confine reuses it.

3. Root-privileged bind mount When snap-confine initializes a new sandbox, it may bind-mount the attacker-controlled directory with elevated privileges, effectively executing attacker-supplied content in a root context.

This combination of predictable cleanup behavior and insufficient validation leads to a classic privilege escalation scenario, where user-controlled input is indirectly introduced into a privileged execution path.

Affected Software:

- Ubuntu Desktop 24.04 LTS: snapd versions prior to 2.73+ubuntu24.04.2.
- Ubuntu Desktop 25.10 LTS: snapd versions prior to 2.73+ubuntu25.10.1.
- Ubuntu 26.04 LTS (Dev): snapd versions prior to 2.74.1+ubuntu26.04.1.
- Upstream snapd: Versions prior to 2.75.

(Note: Older versions like 16.04, 18.04, 20.04, and 22.04 LTS are not vulnerable in their default configurations, but Canonical has issued hardening patches for them regardless. Also server installations are not affected since systemd-tmpfiles-clean.timer is not enabled by default).

Real-World Exploitation Scenario

This vulnerability is essentially a TOCTOU race condition taking advantage of already privileged system components. Here's how it might go down:

1. An attacker only needs a low-privileged account on an Ubuntu system. The attacker plants a script to monitor the /tmp directory.

2. The script waits silently for 30 days (on Ubuntu 24.04) or 10 days (on Ubuntu 25.10) for the system to run its automated systemd-tmpfiles cleanup routine.

3. The moment the directory is cleared, the attacker's script intercepts the process, plants the malicious files, and triggers the snapd sandbox, walking away with a persistent root shell.

What is the Impact?

This vulnerability represents a significant internal threat for organizations relying on Linux desktops:

- Massive Attack Surface: Snapd is installed and active by default on millions of Ubuntu Desktop installations.
- Zero User Interaction Required: The exploit does not require the attacker to phish an administrator or wait for a user to type a sudo password; the system's own automated cleanup processes trigger the vulnerability.
- Total Compromise: Successful exploitation gives the attacker complete read/write access to every file on the system, the ability to deploy persistent backdoors, and the opportunity to pivot to deeper network resources. Any shared Ubuntu desktop environment must be treated as potentially compromised if unpatched.

Mitigation

- Update snapd to the latest patched version provided by your distribution
- Apply all vendor security updates promptly (Ubuntu has already released fixes)
- Monitor and restrict local user access where possible
- Audit systems for unusual activity around temporary directories such as /tmp/.snap

Snapped - Machine

To offer a chance to explore these vulnerabilities in practice, we've whipped up Snapped. This is a Hard Difficulty machine that will allow you to experience and chain these vulnerabilities to reach root.

You will:

- Exploit CVE-2026-27944 to access and decrypt an Nginx-UI backup
- Extract and crack encrypted user credentials
- Abuse CVE-2026-3888 to reach root by exploiting a race condition between snap-confine (SUID) and systemd-tmpfiles

Final Thoughts

Both of these are a vulnerabilities are a good reminder that you don’t always need something flashy or overly complex to break a system - sometimes all it takes is a couple of oversights hidden in the right place.

In Nginx-UI, a sensitive endpoint is left open and then it's paired with encryption that looks reassuring until you realize the keys are being handed out with the data. Neither oversight is trivial on its own but together, they turn a bad situation into a critical one.

snapd is a bit sneakier. Nothing is obviously “wide open” but the way temporary directories are cleaned up and reused creates a tiny window of opportunity. An attacker just has to be in the right place at the right time, and suddenly routine system behavior turns into a path to root.

Neither bug relies on especially complex exploitation. That’s what makes them dangerous. It's not easy to break strong security controls but it sure is fun to find the places where they aren't properly enforced in the first place.

Latest articles

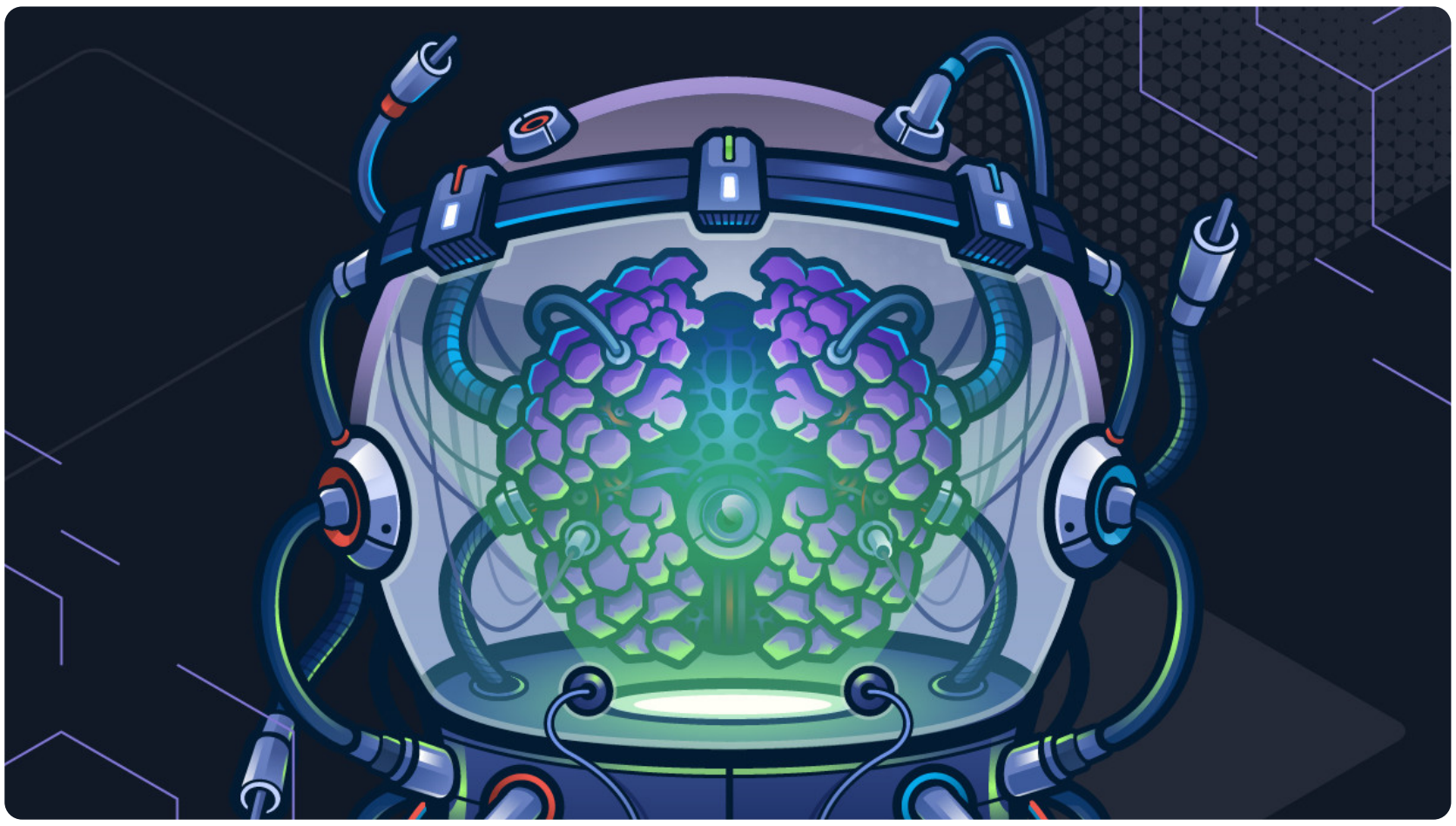
View all

The latest news and updates, direct from Hack The Box

Hack The Box wins 2026 industry awards as it builds the future of cyber readiness

News 4 min read

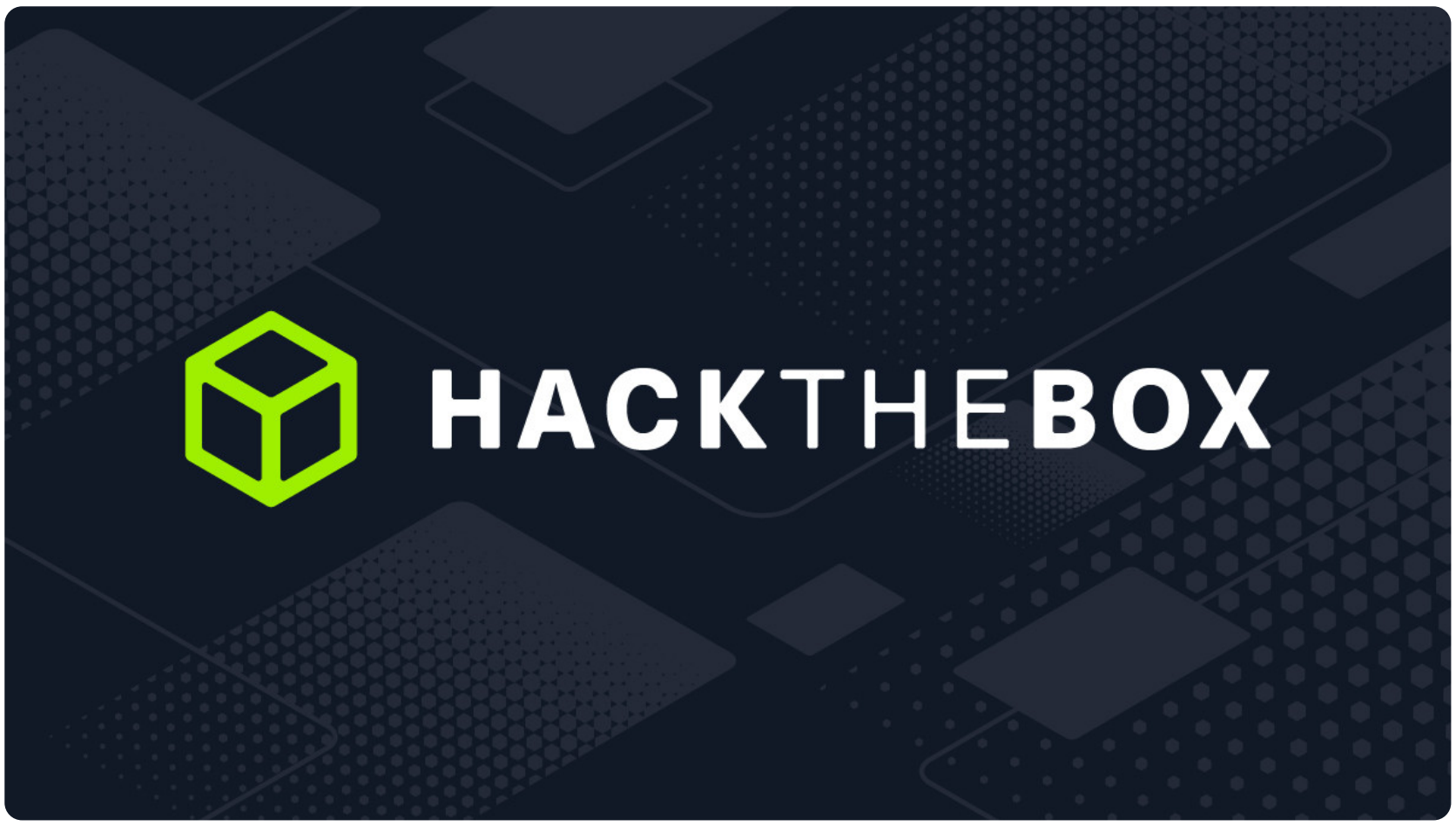
SportyBartlett Apr 16, 2026



Be the human element in AI hacking with HTB COAE

Red Teaming 4 min read

JXoaT Apr 02, 2026



Hack The Box named a Leader in Independent Cybersecurity Skills and Training Platform Evaluation

News 4 min read

Ophie Mar 11, 2026

Receive our weekly blog digest

First name*

Last name*

Enter your email to subscribe.*

Submit

The #1 platform to build attack-ready teams and organizations.

Get a demo

FORRESTER® WAVE LEADER

CYBER ESSENTIALS

ISO 27001 ISO 27701 ISO 9001

G2 4.8 Capterra 4.8

Products

Teams

Courses & Certifications

Cyber Ranges

Enterprise Attack Simulations

Cloud Infrastructure Simulations

Capture The Flag

Tabletop Exercises

Talent Sourcing

Individuals

Solutions

Job Roles

Red Teams

Blue Teams

Purple Teams

Industries

Government

Education

Finance

MSSPs

Resources

Community

Blog

Industry Reports

Webinars

AMAs

Learn with HTB

Customer Stories

Cheat Sheets

Compliance Sheets

Glossary

Company

About us

Newsroom

Careers

Brand Guidelines

Certificate Validation

Trust Center

Vulnerability Disclosure

Product Updates

Status

© 2026 Hack The Box